

Rapport de projet

Simulation de jeu et analyse stratégique

Travail réalisé par :

Mansour Bengoumi

Yasser Bakkali

1. Architecture et choix de conception

1.1 Structure des fichiers

Le projet est organisé en modules Python indépendants, chacun ayant une responsabilité unique et clairement définie. Cette séparation favorise la lisibilité, la testabilité et la maintenabilité du code.

Voici le rôle de chaque fichier :

- **main.py** : point d'entrée principal, dispatche vers les trois modes d'exécution (play, simulate, explore).
- **map.py** : lecture, validation et représentation de la grille.
- **rules.py** : lecture, validation et vérification des paramètres de jeu.
- **moves.py** : lecture et validation de la séquence de mouvements.
- **engine.py** : moteur de jeu contenant les fonctions `init_state`, `apply_action` et `is_terminal`.
- **simulate.py** : simulations aléatoires et calcul des statistiques.
- **explore.py** : exploration récursive de l'arbre des mouvements avec mémorisation.
- **utils.py** : fonctions utilitaires partagées, notamment l'écriture du fichier `run_log.txt`.

1.2 Représentation de l'état du jeu

L'état du jeu à un instant donné est représenté par un dictionnaire Python contenant les champs suivants :

```
state = {  
    "pos" : (ligne, colonne), # position courante du joueur  
    "score" : 0, # score courant  
    "steps" : 0, # nombre de pas effectuées  
    "collected" : set(), # coordonnées des bonus déjà collectés  
    "hp" : HP, # points de vie (uniquement en mode health)  
    "status" : "CONTINUE" # CONTINUE | SUCCESS | FAILURE | LIMIT  
}
```

Ce choix permet de passer l'état facilement entre fonctions, de le copier rapidement avec `copy.deepcopy()` lors de l'exploration récursive, et de l'inspecter simplement lors du débogage.

1.3 Choix de conception clés

- **Immuabilité de l'état** : la fonction `apply_action()` effectue toujours un `copy.deepcopy()` avant toute modification. Cela garantit l'absence d'effets de bord entre les branches de l'arbre récursif et entre les simulations indépendantes.
- **Gestion des obstacles** : une tentative de mouvement vers une case X ou hors grille est comptabilisée comme un pas (coût `MOVE_COST` déduit) mais la position du joueur reste inchangée.
- **Bonus à usage unique** : l'ensemble `collected` mémorise les coordonnées des cases B déjà visitées. Un second passage sur la même case B ne rapporte aucun point supplémentaire.
- **Validation stricte des entrées** : chaque fichier est intégralement validé avant toute exécution (symboles autorisés, dimensions cohérentes, paramètres obligatoires présents).

2. Modes d'exécution

2.1 Mode play

Ce mode rejoue de façon déterministe la séquence d'actions contenue dans le fichier moves.txt. Chaque action est appliquée via `apply_action()` et une ligne de journal est produite et affichée. L'exécution s'arrête dès qu'un état terminal est atteint, c'est-à-dire SUCCESS, FAILURE ou LIMIT.

La commande d'exécution est la suivante :

```
python main.py play map.txt rules.txt moves.txt
```

Exemple de sortie sur la carte de test 1 :

```
Step 1 | Action : RIGHT | Pos : (4, 1) | Score : -1 | Event : NONE
Step 2 | Action : RIGHT | Pos : (4, 2) | Score : -2 | Event : NONE
Step 3 | Action : UP    | Pos : (3, 2) | Score : -3 | Event : NONE
Step 4 | Action : UP    | Pos : (2, 2) | Score : 6  | Event : BONUS (+10)
RESULT : LIMIT | Steps : 7 | Final Score : 3
```

2.2 Mode simulate

Ce mode effectue SIMU_COUNT simulations indépendantes avec des actions tirées aléatoirement parmi les cinq actions valides. La graine RANDOM_SEED garantit la reproductibilité des résultats d'une exécution à l'autre. À l'issue de toutes les simulations, un rapport statistique complet est affiché.

La commande d'exécution est la suivante :

```
python main.py simulate map.txt rules.txt
```

2.3 Mode explore

Ce mode explore récursivement l'arbre de tous les chemins possibles jusqu'à la profondeur DEPTH_EXPLORATION. Il retourne la séquence d'actions menant au score maximal ainsi que le nombre total de feuilles explorées après élagage.

La commande d'exécution est la suivante :

```
python main.py explore map.txt rules.txt
```

3. Algorithme d'exploration récursive

3.1 Principe général

L'exploration récursive construit implicitement un arbre dont chaque nœud représente un état du jeu et chaque arc une action. La racine est l'état initial, c'est-à-dire la position P avec un score égal à 0. À chaque niveau, les cinq actions possibles sont testées. Sans optimisation, la complexité théorique est en $O(5^d)$ où d est la profondeur maximale.

3.2 Pseudo-algorithme

```
fonction explore(etat, profondeur, chemin, meilleur_ref, memo):
  Si profondeur == 0 OU etat terminal :
    mettre à jour meilleur_ref si etat.score > meilleur
    RETOURNER 1
  empreinte = (pos, collected_frozen, steps)
  Si empreinte dans memo ET memo[empreinte] >= etat.score :
    RETOURNER 0 (branche elaguee)
  memo[empreinte] = etat.score
  total = 0
  POUR chaque action DANS [UP, DOWN, LEFT, RIGHT, WAIT] :
    prochain = apply_action(etat, action)
    total += explore(prochain, profondeur - 1, ...)
  RETOURNER total
```

3.3 Optimisations mises en oeuvre

Sans optimisation, une profondeur de 12 génère environ 244 millions de nœuds, ce qui est inexploitable en temps raisonnable. Deux stratégies d'élagage ont été mises en oeuvre.

Arrêt anticipé sur état terminal : dès qu'un état SUCCESS ou FAILURE est atteint, la récursion s'arrête immédiatement car il est inutile de continuer à explorer sous un état déjà terminé.

Mémoïsation : un dictionnaire memo associe chaque empreinte d'état (position, bonus collectés, nombre de pas) au meilleur score depuis lequel cet état a été atteint. Si une branche arrive dans un état déjà visité avec un score inférieur ou égal, elle est abandonnée car elle ne peut pas produire de meilleur résultat.

Les gains observés sur la carte de test 1 sont les suivants. Pour une profondeur de 6, le nombre de feuilles passe de 15 625 à 105, soit un gain d'un facteur 149. Pour une profondeur de 10, il passe de 9 765 625 à 289 feuilles, un gain de facteur 33 800. Pour une profondeur de 12, le nombre théorique de 244 millions de nœuds est réduit à 315 feuilles, soit un gain de facteur 775 000.

3.4 Résultat sur la carte de test 1

=== Exploration recursive (profondeur max : 12) ===

Meilleur score trouve : 51

Sequence optimale : UP UP RIGHT RIGHT RIGHT RIGHT RIGHT DOWN RIGHT

Feuilles explorees : 315

Le score de 51 correspond à la collecte du bonus B (+10) puis à l'atteinte de la cible T (+50), diminué de 9 coups ayant chacun un coût de 1, soit $50 + 10 - 9 = 51$.

4. Analyse des résultats de simulation

4.1 Configuration utilisée

Les simulations ont été effectuées sur la carte de test 1, une grille de 5 lignes et 7 colonnes, avec les paramètres suivants : MAX_MOVES = 50, MOVE_COST = 1, BONUS_POINTS = 10, TARGET_BONUS = 50, ENEMY_MODE = death, SIMU_COUNT = 200 et RANDOM_SEED = 0.

4.2 Rapport obtenu

```
=== Rapport de simulation (200 parties) ===  
Taux de succes : 15.00 %  
Score moyen   : -33.72  
Score max     : 47  
Score min     : -50  
Ecart-type    : 23.59  
Pas moyens    : 47.8  
Distribution   : SUCCESS=30 | FAILURE=0 | LIMIT=170
```

4.3 Interprétation

Taux de succès faible (15 %) : un joueur aléatoire a peu de chances d'atteindre la cible dans une grille parsemée d'obstacles. La cible T se situe dans un coin de la grille, difficile d'accès sans stratégie dirigée.

Score moyen négatif (-33.72) : la majorité des parties atteignent la limite de pas sans succès, soit 170 parties sur 200. Chaque pas coûte 1 point, donc une partie de 50 pas sans bonus rapporte -50. Le score moyen est tiré vers le bas par ces parties.

Score maximal égal à 47 : ce score correspond à un succès (+50) avec collecte du bonus (+10) en 13 pas (-13), soit 47 au total. Certaines simulations trouvent le chemin optimal par chance.

Ecart-type élevé (23.59) : la dispersion des scores reflète la grande variabilité du comportement aléatoire. Certaines parties réussissent rapidement, d'autres errent jusqu'à la limite de pas.

FAILURE égal à zéro : la carte de test 1 ne contient pas d'ennemis malgré le mode death, donc aucune partie ne se termine par une mort.

5. Difficultés rencontrées et solutions adoptées

5.1 Explosion combinatoire de l'exploration

Sans optimisation, une profondeur de 12 génère environ 244 millions de nœuds, ce qui est inexploitable dans un temps raisonnable. La mémoïsation réduit ce nombre à quelques centaines de nœuds significatifs sur les cartes de test, avec un gain de facteur 775 000 observé sur la carte de test 1.

5.2 Effets de bord dans la récursion

Les premières versions du code modifiaient l'état en place, provoquant des comportements imprévisibles entre branches. La solution adoptée est un appel systématique à `copy.deepcopy()` dans `apply_action()`, conformément aux préconisations du sujet.

5.3 Gestion des bonus à usage unique

Un bonus revisité ne doit pas rapporter de points supplémentaires. L'ensemble `collected` dans l'état mémorise les coordonnées (r, c) des cases B déjà visitées. Cette structure est copiée profondément à chaque appel à `apply_action()`.

5.4 Cohérence des collisions

Un mouvement bloqué par un obstacle ou par les bords de la grille consomme un pas et coûte `MOVE_COST`, mais la position du joueur reste inchangée. Ce comportement a été documenté et testé explicitement dans le bloc de test du moteur de jeu.

5.5 Validation stricte des fichiers d'entrée

Les fichiers peuvent contenir des espaces multiples, des commentaires, des lignes vides ou des paramètres inconnus. L'utilisation de `line.split()` sans argument gère naturellement les espaces multiples. Chaque paramètre est vérifié contre la liste des paramètres autorisés dès le chargement.

6. Améliorations et extensions proposées

6.1 Elagage implémenté

La mémorisation décrite en section 3.3 constitue l'optimisation principale du mode explore. Elle a été intégrée directement dans la fonction `explore()` du module `explore.py` et rend le mode explore utilisable pour des profondeurs allant jusqu'à une vingtaine de niveaux.

6.2 Extensions envisageables

- **Visualisation de la grille** : affichage textuel de la grille avec la position du joueur, les bonus restants et les ennemis, mise à jour à chaque étape du mode play.
- **Interface interactive** : mode interactif permettant au joueur de saisir des actions au clavier en temps réel, avec affichage de la grille mis à jour.
- **Tests unitaires** : formaliser les blocs de test existants en tests pytest avec des assertions précises et des cas limites tels qu'une grille de taille 1x1, `MAX_MOVES` égal à 0 ou un joueur démarrant sur une case B ou E.
- **Exploration heuristique** : remplacer la recherche exhaustive par un algorithme A* utilisant la distance de Manhattan à la cible T comme heuristique, afin de trouver rapidement de bonnes solutions sur de grandes grilles.